

Windowing Overview

- Relevance
- Main lobe/side lobe tradeoff
- Popular windows
- Examples

Windowing

- In practice, we cannot observe a signal $x(n)$ for $n = -\infty$ to $n = \infty$
- Thus we must truncate, or *window*, the signal: $x(n) \cdot w(n)$
- Windowing a DT signal has a similar effect to windowing a CT signal
- In the frequency domain
 - Pure tones (sinusoids) change from impulses to the DTFT of the window function
 - The spectrum is *blurred*
- These are both undesirable effects
- It is crucial that you understand them
- Misunderstanding them will lead to wrong conclusions about spectral content

DFT Interpretation: Windowing

If $x(n)$ is infinite duration and $w(n)$ is a finite duration window, say

$$w(n) = \begin{cases} 1 & 0 \leq n < N \\ 0 & \text{otherwise} \end{cases}$$

Then the product $x(n)w(n)$ is a discrete-time product and we know

$$x(n)w(n) \xleftrightarrow{\mathcal{FT}} \frac{1}{2\pi} \int_{2\pi} X(e^{ju})W(e^{j(\omega-u)}) du$$

- In words, the sampled signal's spectrum is equal to the convolution of the signal's spectrum $X(e^{j\omega})$ and the window's spectrum $W(e^{j\omega})$
- This will be very important when we discuss spectral estimation

Example 1: Windowing

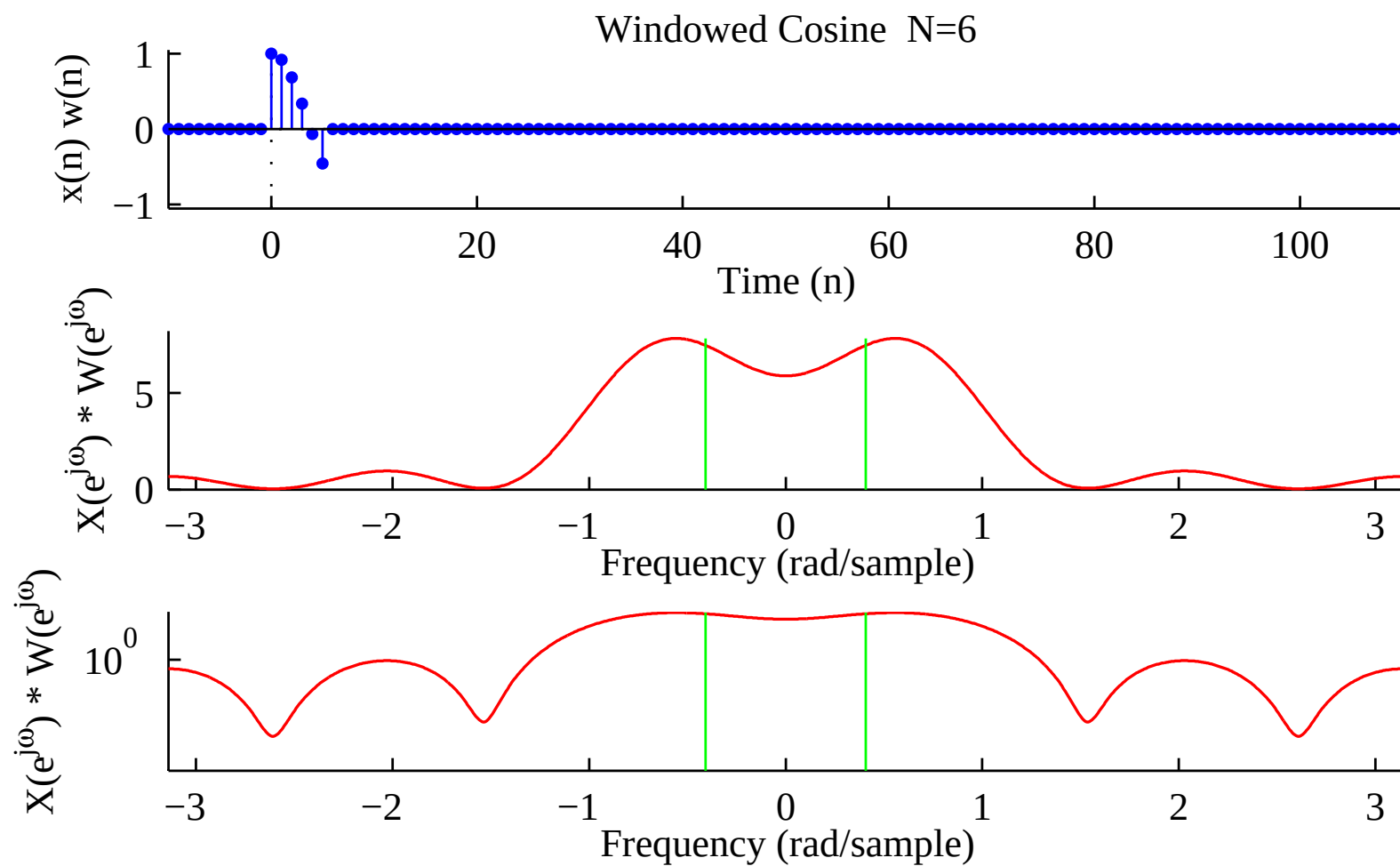
Consider a pure sinusoidal tone $x(n) = \cos(\omega_o n)$. Find the Fourier transform of $x(n) \cdot w(n)$ where $w(n)$ is a rectangular window.

Plot the true transform and estimated transform for $\omega_o = 0.13\pi$ radian per sample and $N = 6, 10, 50$ and 100 .

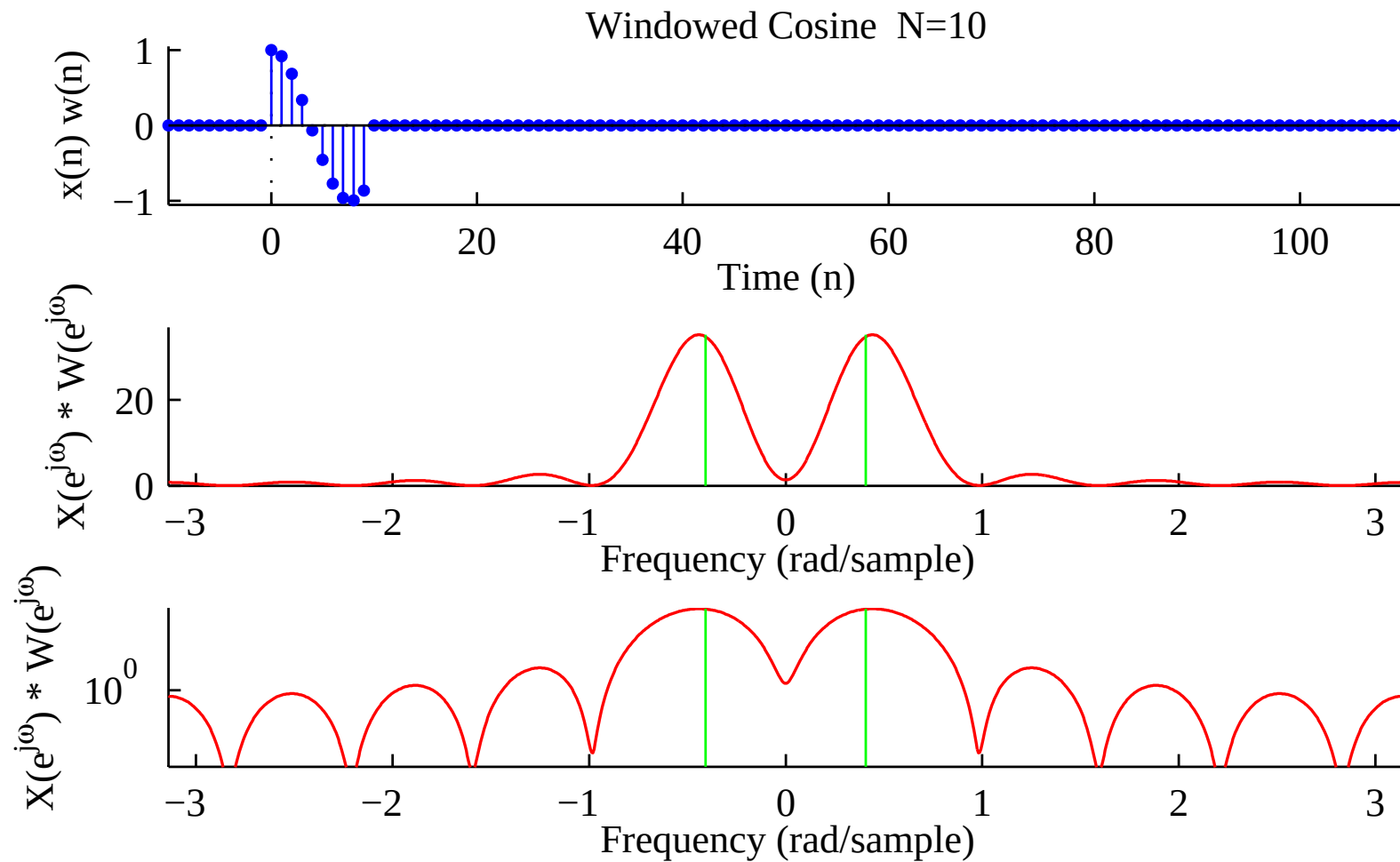
$$e^{j\omega_o n} \Leftrightarrow 2\pi \sum_{\ell=-\infty}^{\infty} \delta(\omega - \omega_o - 2\pi\ell)$$

$$w(n) \Leftrightarrow \frac{\sin(\omega N/2)}{\sin(\omega/2)} e^{-j\omega(N-1)/2}$$

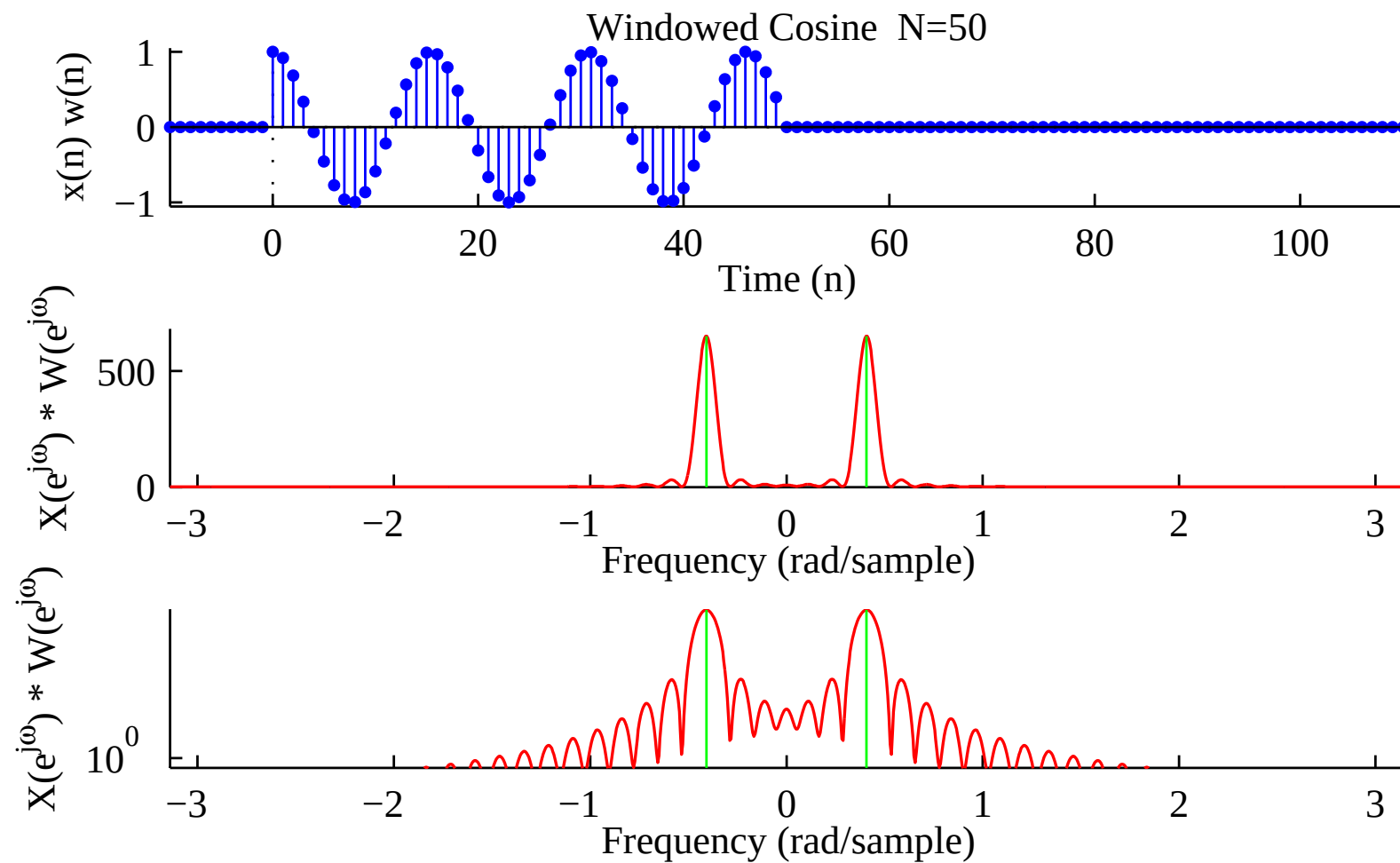
Example 1: $N = 6$



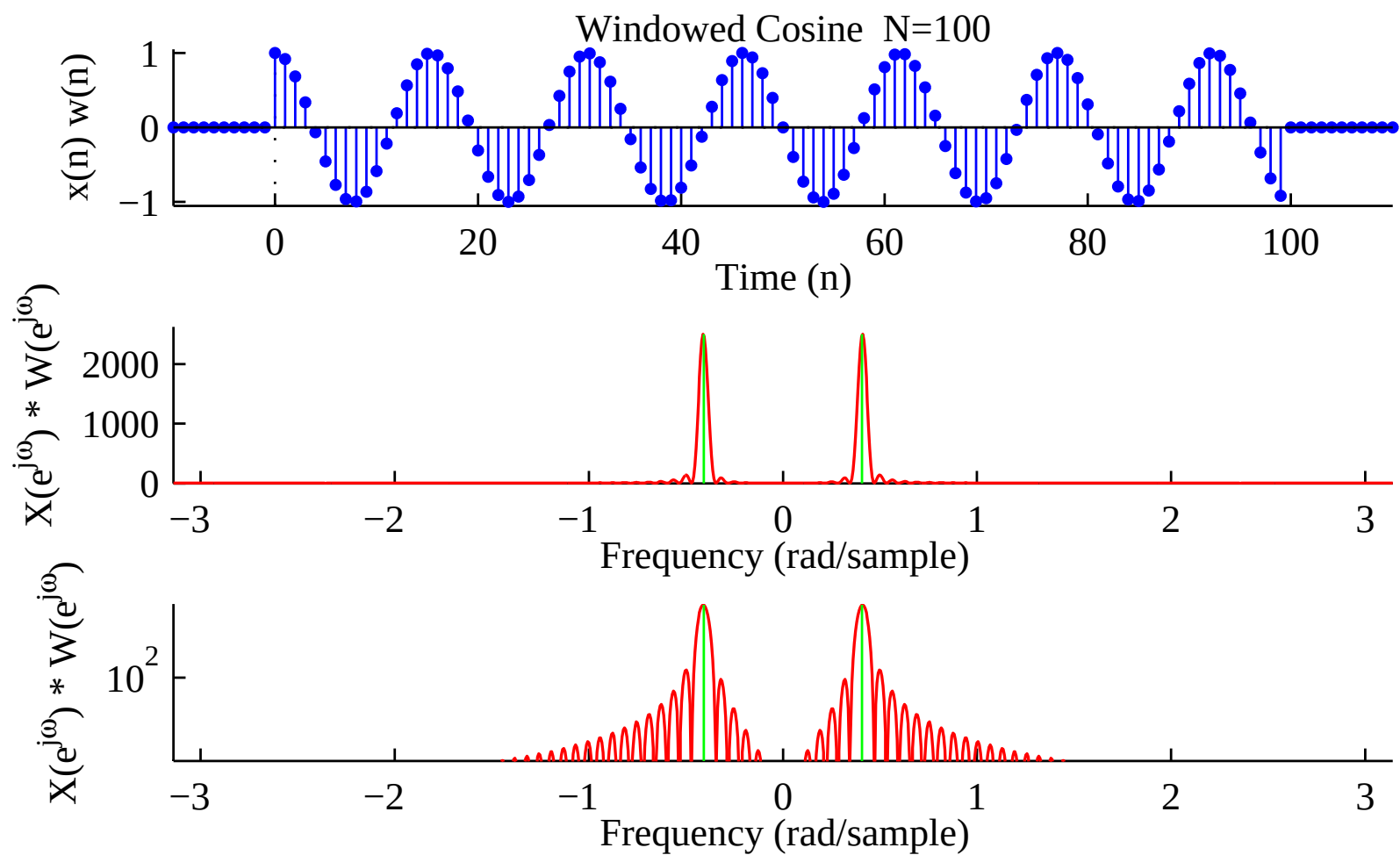
Example 1: $N = 10$



Example 1: $N = 50$



Example 1: $N = 100$



Example 1: MATLAB Code

```
%function [] = Windowing();
close all;
clear;

N = [6 10 50 100];
n = -10:(max(N)+10);
nz = 2^10; % Number of zeros for padding
k = (1:nz/2+1); % Frequency indices
w = 2*pi*(k-1)/nz; % Frequencies (rads/sample)
w = [-fliplr(w) w(2:end)]; % Include negative frequencies
wo = 0.13*pi; % Frequency of the sinusoid

for cnt = 1:length(N),
    %wn = (n>=0 & n<N);
    wn = (n>=0 & n<N(cnt));
    x = cos(wo*n).*wn;
    X = fft(x,nz);
    Rx = abs(X(k)).^2;
    Rx = [fliplr(Rx) Rx(2:end)]; % Include negative frequencies

    figure;
    FigureSet(1,'LTX');
    subplot(3,1,1);
    h = stem(n,x,'filled');
    set(h(1),'MarkerFaceColor','b');
    set(h(1),'MarkerSize',2);
    xlim([min(n) max(n)]);
```

```

        ylim([-1.05 1.05]);
        AxisLines;
        xlabel('Time (n)');
        ylabel('x(n) w(n)');
        title(sprintf('Windowed Cosine N=%d',N(cnt)));
        box off;
    subplot(3,1,2);
        h = plot(w,Rx,'r','LineWidth',0.6);
        xlim([-pi pi]);
        ylim([0 max(Rx)*1.05]);
        hold on;
            h = plot(wo*[1 -1;1 -1],[0 0;1 1]*max(Rx),'g');
            hold off;
        xlabel('Frequency (rad/sample)');
        ylabel('X(e^{j\omega}) * W(e^{j\omega})');
        box off;
    subplot(3,1,3);
        h = semilogy(w,Rx,'r','LineWidth',0.6);
        xlim([-pi pi]);
        ylim(max(Rx)*[0.001 1.05]);
        hold on;
            h = plot(wo*[1 -1;1 -1],[0.001 0.001;1 1]*max(Rx),'g');
            hold off;
        xlabel('Frequency (rad/sample)');
        ylabel('X(e^{j\omega}) * W(e^{j\omega})');
        box off;
    AxisSet(8);
    print(sprintf('Windowing%03d',N(cnt)),'-depsc');
end;

```

Windowing

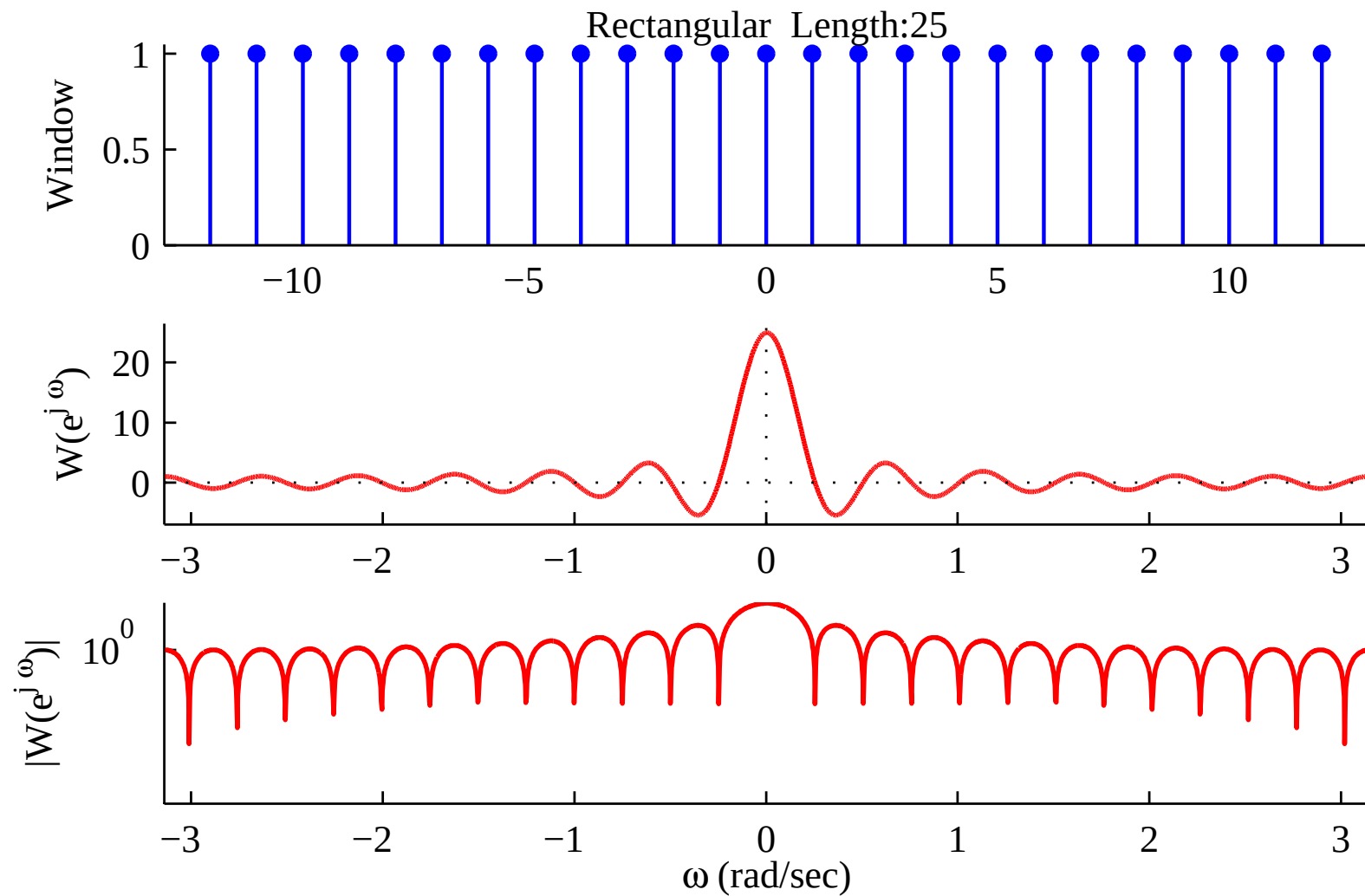
$$x(n)w(n) \xLeftrightarrow{\mathcal{FT}} \frac{1}{2\pi} \int_{-\pi}^{\pi} X(e^{ju})W(e^{j(\omega-u)}) du$$

- Windowing in the time domain is equivalent to convolution in the frequency domain
- Would like window to be as close as a Dirac impulse as possible
- Length of window is constrained to N samples
- Key tradeoff: main lobe width an amplitude of side lobes
 - Both can cause *bias* in the frequency domain
- There are many windows to chose from
- Rectangular window has smallest main lobe, but largest side lobes
- Two types of windows
 - **Fixed:** defined only by the duration of the window, N
 - **Parametric:** have parameters that control the tradeoff between main lobe width and side lobe amplitude

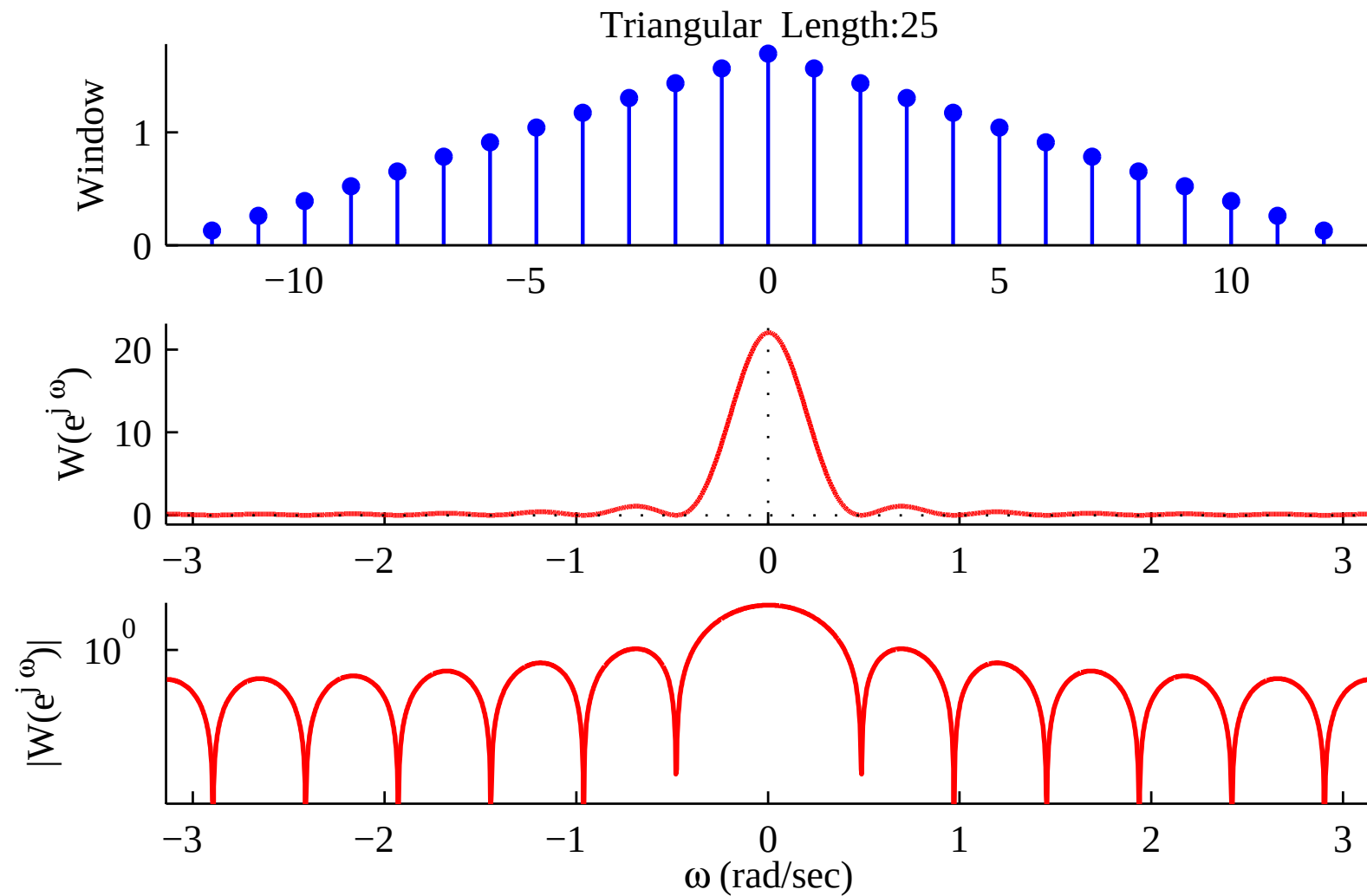
Example 2: Fixed Windows

Plot various fixed windows and their Fourier transform. Center the windows so that they have even symmetry and length $N = 25$ samples.

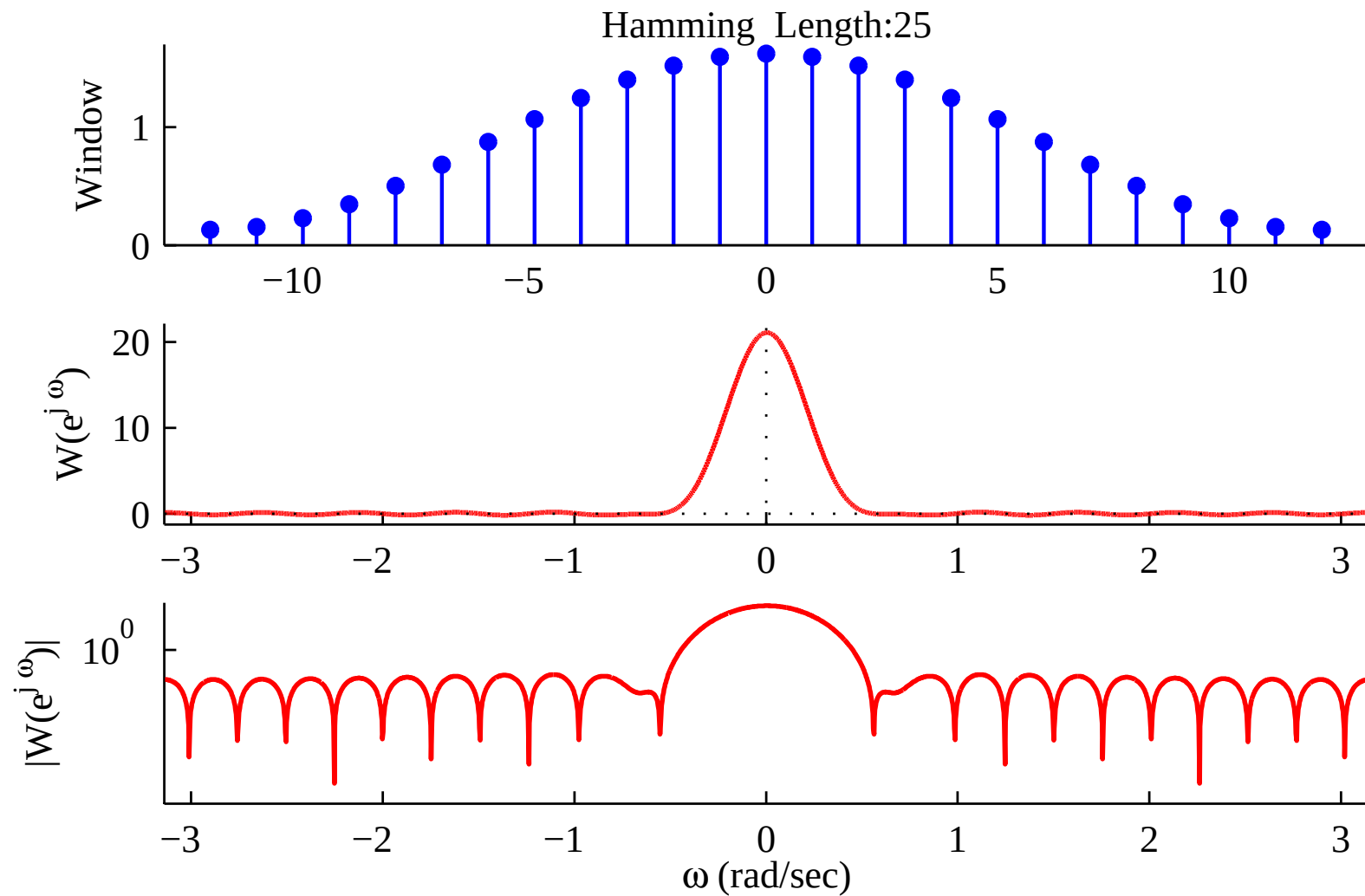
Example 2: Rectangular



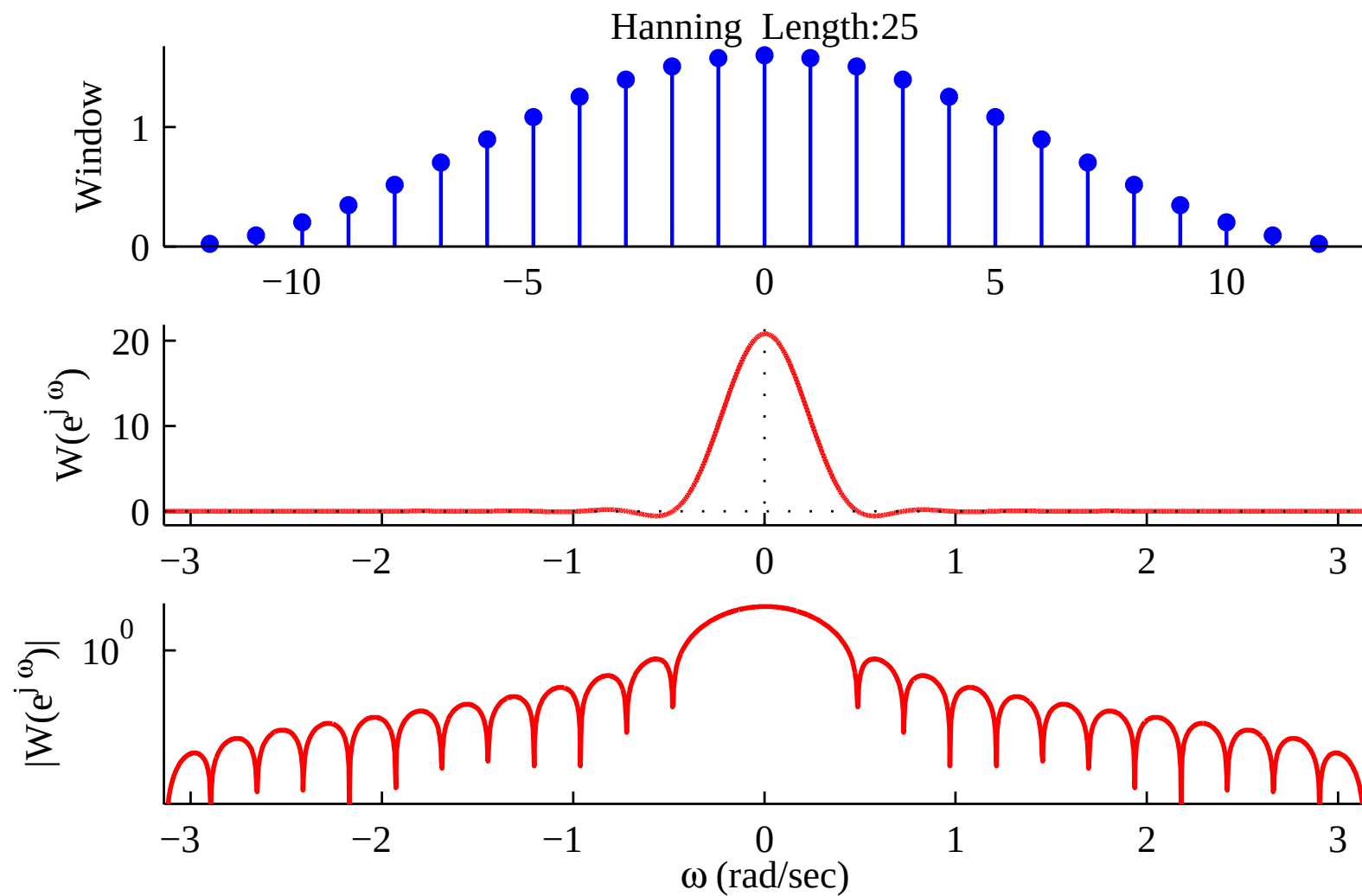
Example 2: Triangular



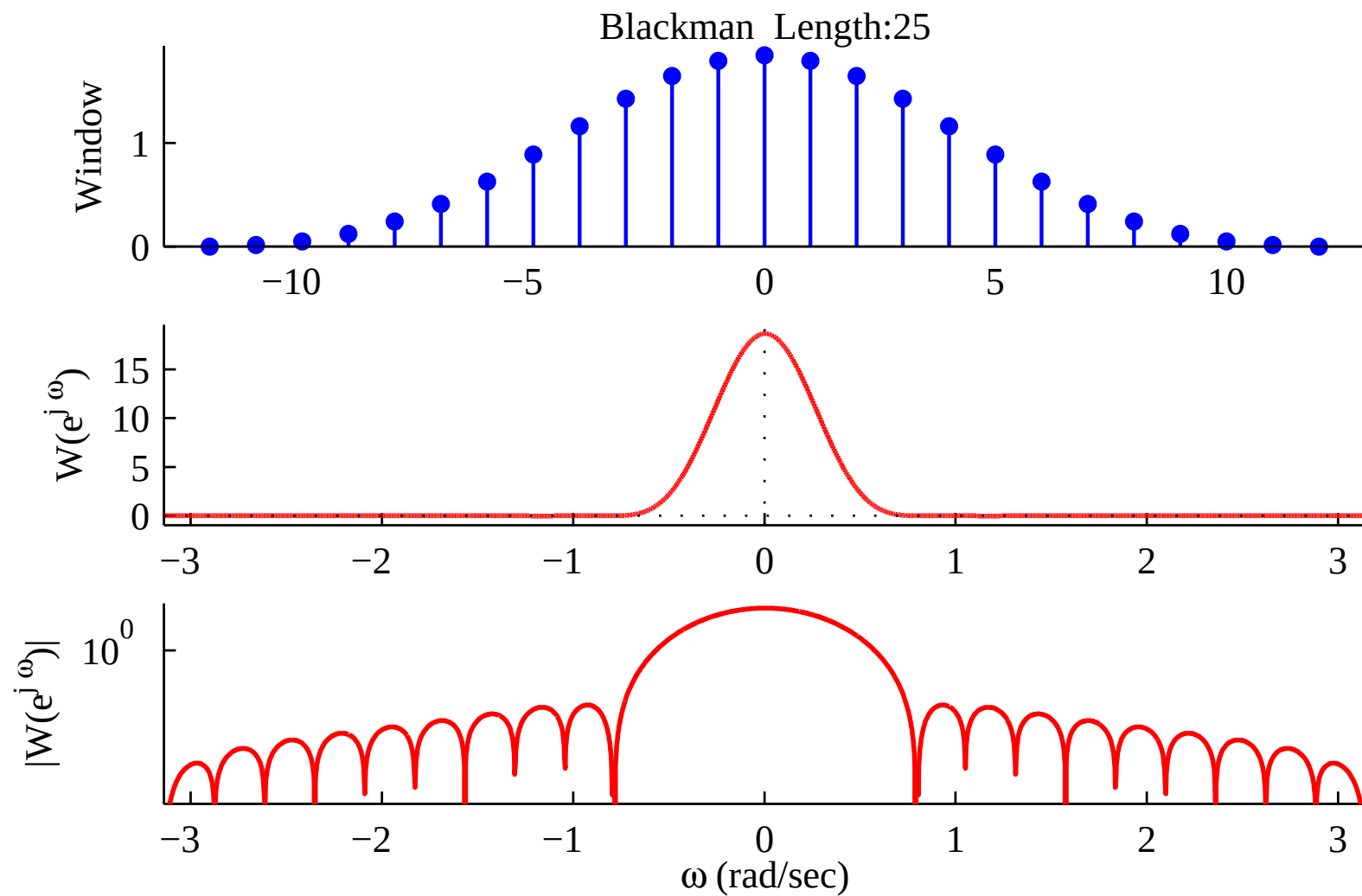
Example 2: Hamming



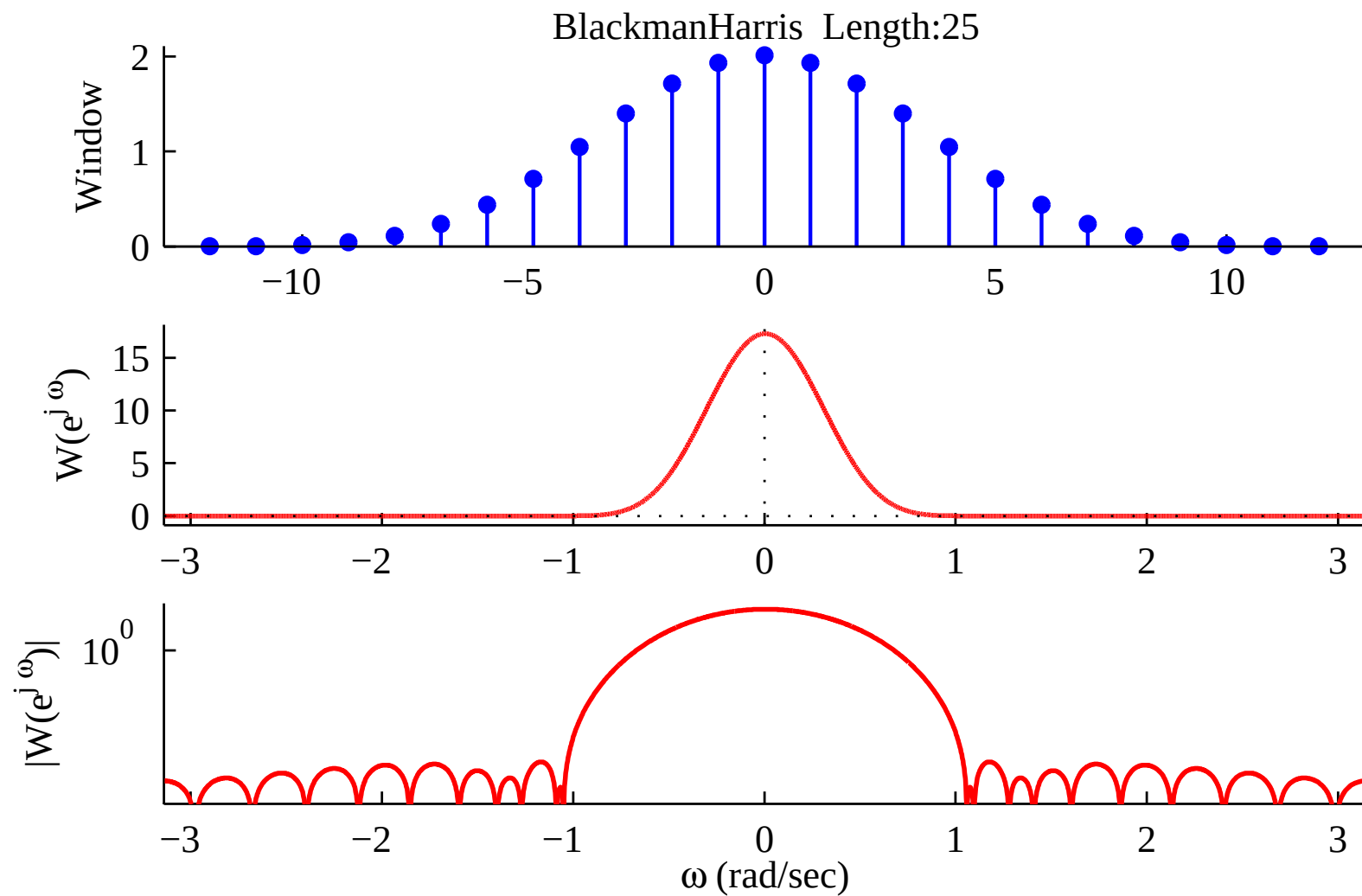
Example 2: Hanning



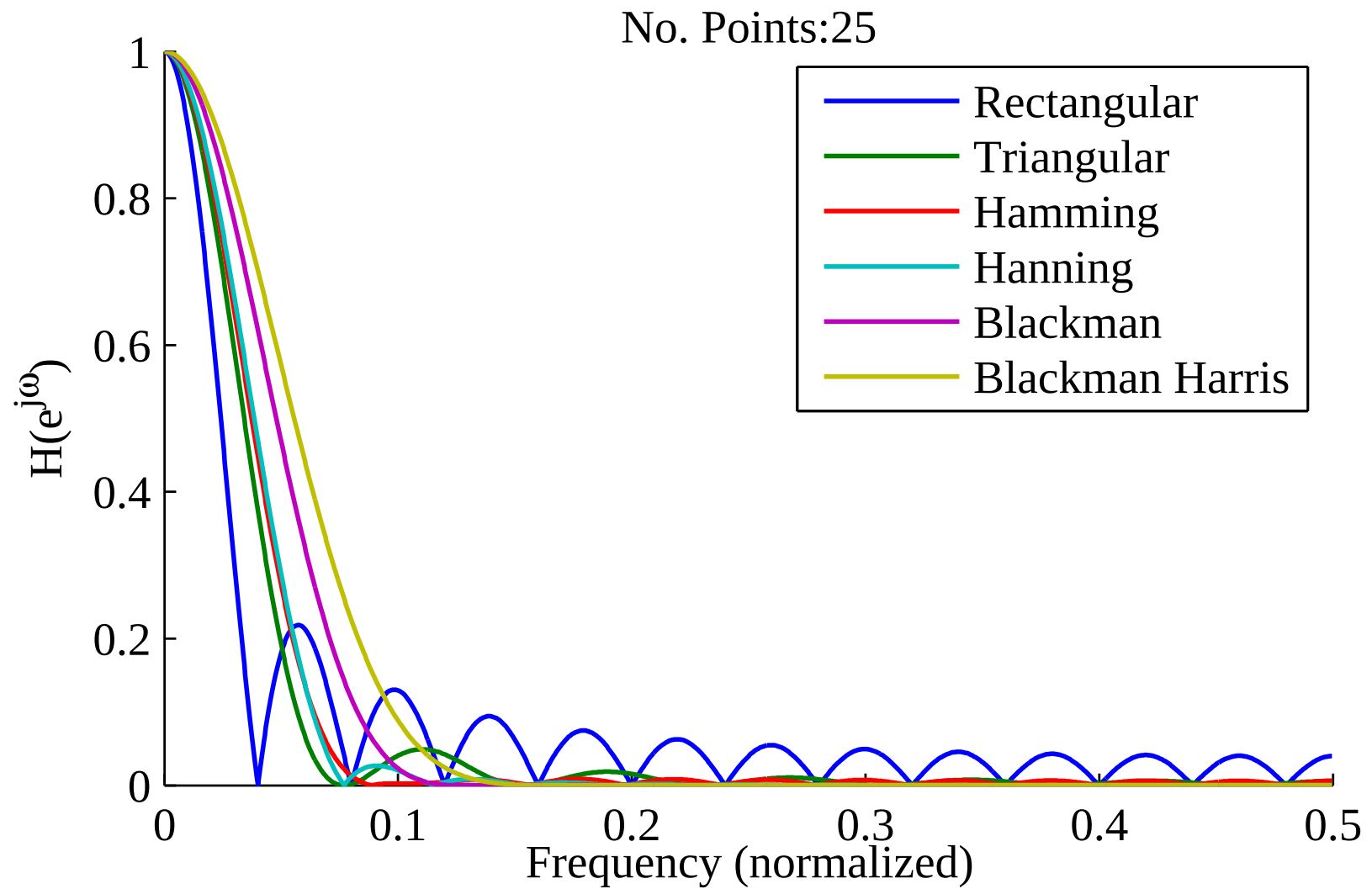
Example 2: Blackman



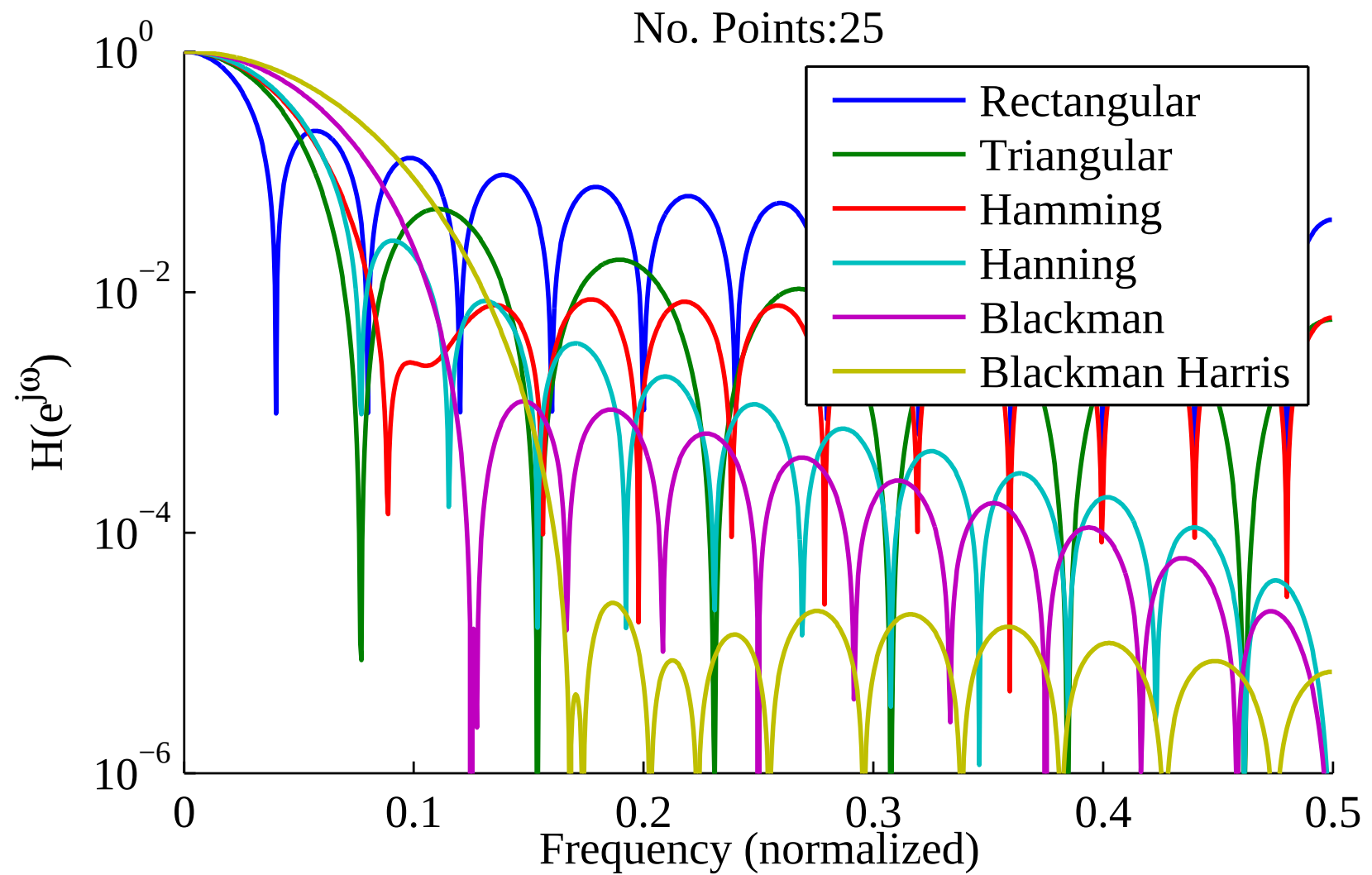
Example 2: Blackman-Harris



Example 2: All



Example 2: All



Example 2: MATLAB Code

```
%function [] = Windows;
close all;
clear;
N = 25;
M = 2048;

%=====
% Main Loop
%=====
n = -(N-1)/2:(N-1)/2;
for c1 = 1:6,
    switch c1,
        case 1,
            wn = rectwin(N);
            stW = 'Rectangular';
        case 2,
            wn = triang(N);
            stW = 'Triangular';
        case 3,
            wn = hamming(N);
            stW = 'Hamming';
        case 4,
            wn = hanning(N);
            stW = 'Hanning';
        case 5,
            wn = blackman(N);
            stW = 'Blackman';
```

```

        case 6,
            wn = blackmanharris(N);
            stW = 'BlackmanHarris';
        end;

wn = wn*sqrt(N/sum(wn.^2));
w = (0:M-1)*pi/(M/2);
W = exp(j*w*(N-1)/2).*fft(wn,M);
W = real(W);
id = [M/2+1:M 1:M/2];
W = W(id);
w = -(M-3)/2:(M+1)/2;
w = w*pi/(M/2);

figure;
FigureSet(1,'LTX');
subplot(3,1,1);
    h = stem(n,wn);
    set(h(1),'MarkerFaceColor','b');
    set(h(1),'MarkerSize',3);
    set(h,'LineWidth',0.8);
    box off;
    ylabel('Window');
    title(sprintf('%s Length:%d',stW,N));
    xlim([-ceil(N/2) ceil(N/2)]);
    ylim([0 1.05*max(wn)]);
subplot(3,1,2);
    h = plot(w,W,'r');
    set(h,'LineWidth',1.0);
    ylabel('W(e^{j \omega})');
    xlim([-pi pi]);

```

```

        ymn = min(W);
        ymx = max(W);
        yrg = ymx-ymn;
        ylim([ymn-0.05*yrg max(W)+0.05*yrg]);
        AxisLines;
        box off;
subplot(3,1,3);
    h = semilogy(w,abs(W),'r');
    set(h,'LineWidth',1.0);
    ylabel('|W(e^{j \omega})|');
    xlabel('\omega (rad/sec)');
    xlim([-pi pi]);
    ylim(N*[1e-6 1.05]);
    box off;
AxisSet(8);
print(['Window' stW],'-depsc');
end;

```

```

%=====
% All
%=====
[hrc,w] = freqz(ones(N,1)/N,1,2^10);
[htr,w] = freqz(triang(N)/sum(triang(N)),1,2^10);
[hhm,w] = freqz(hamming(N)/sum(hamming(N)),1,2^10);
[hhn,w] = freqz(hanning(N)/sum(hanning(N)),1,2^10);
[hbm,w] = freqz(blackman(N)/sum(blackman(N)),1,2^10);
[hbh,w] = freqz(blackmanharris(N)/sum(blackmanharris(N)),1,2^10);

figure;
FigureSet(1,'LTX');
f = w/(2*pi);

```

```

h = semilogy(f,abs(hrc),f,abs(htr),f,abs(hhm),f,abs(hhn),f,abs(hbm),f,abs(hbh));
set(h,'LineWidth',1.0);
ylim([1e-6 1]);
box off;
xlabel('Frequency (normalized)');
ylabel('H(e^{j\omega})');
title(sprintf('No. Points:%d',N));
AxisSet;
legend('Rectangular','Triangular','Hamming','Hanning','Blackman','Blackman Harris');
print -depsc WindowsLog;

figure;
FigureSet(1,'LTX');
f = w/(2*pi);
h = plot(f,abs(hrc),f,abs(htr),f,abs(hhm),f,abs(hhn),f,abs(hbm),f,abs(hbh));
set(h,'LineWidth',1.0);
ylim([0 1]);
box off;
xlabel('Frequency (normalized)');
ylabel('H(e^{j\omega})');
title(sprintf('No. Points:%d',N));
AxisSet;
legend('Rectangular','Triangular','Hamming','Hanning','Blackman','Blackman Harris');
print -depsc Windows;

```

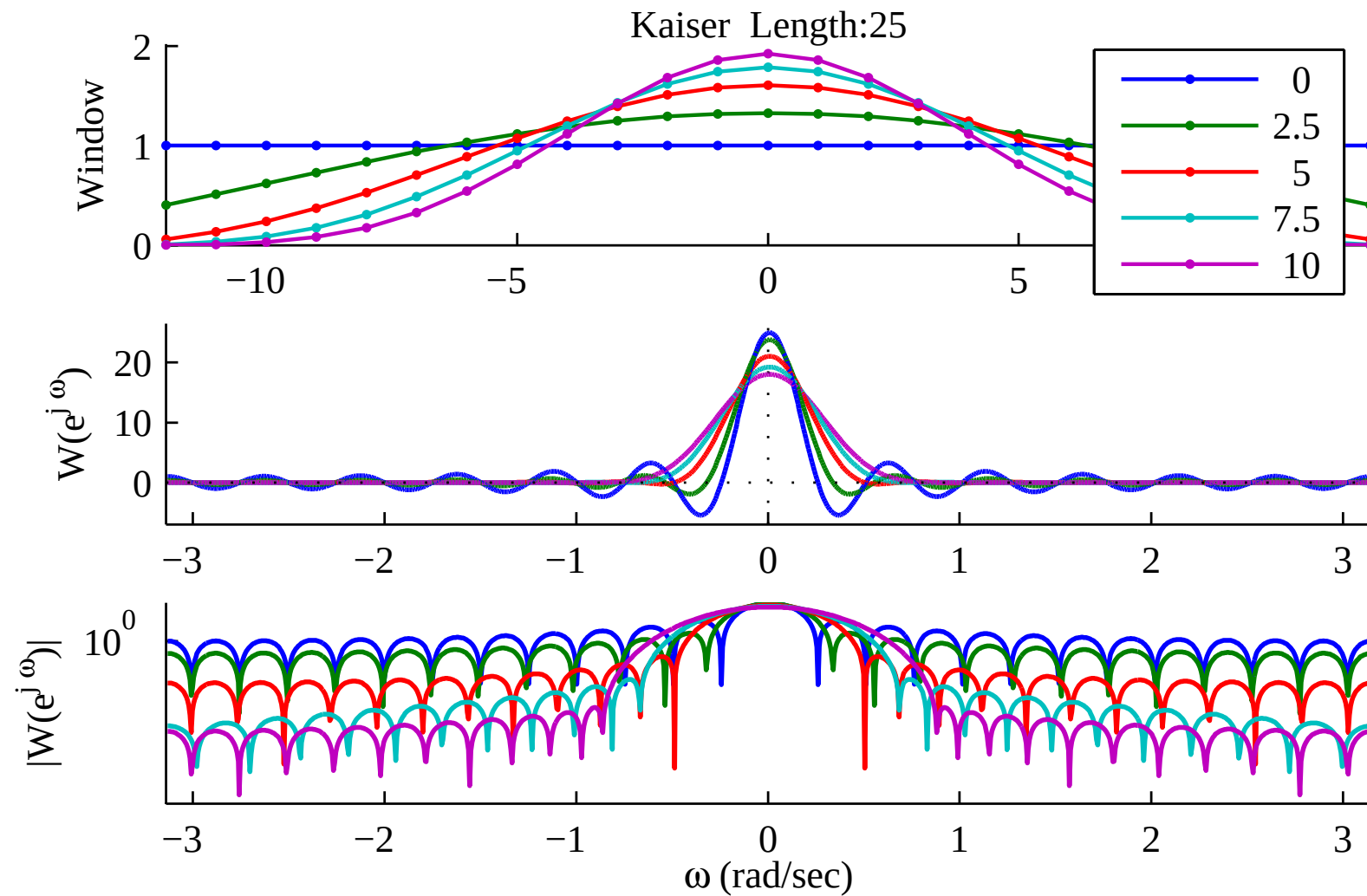
Example 3: Parametric Windows

- There are several parametric windows
- Many of which are included in MATLAB's signal processing toolbox ($\gg$ help window)
- Popular choices include
 - Kaiser: one parameter that controls tradeoff, sidelobes taper
 - Dolph-Chebyshev
 - Tukey (cosine-tapered)
 - Gaussian

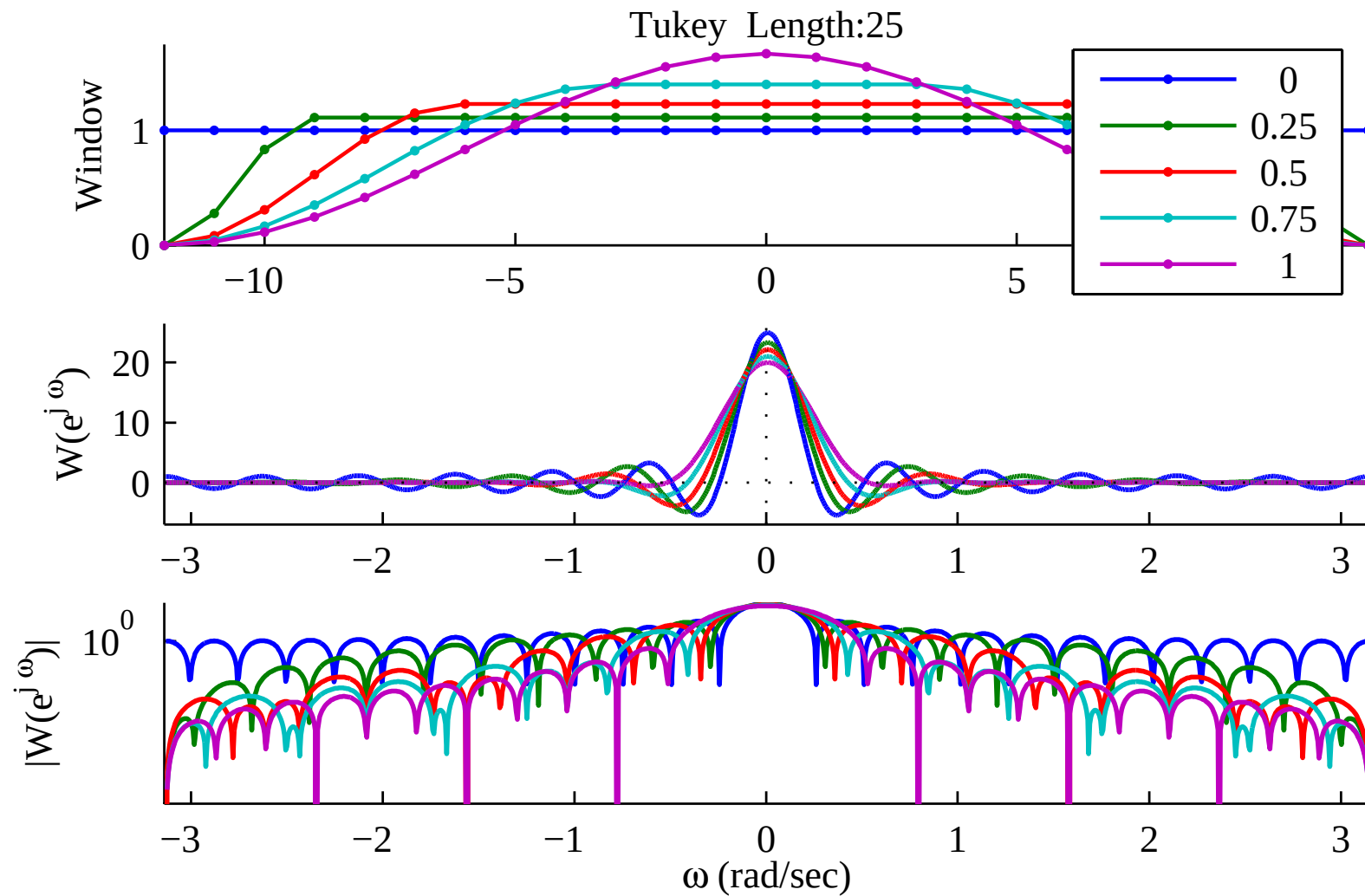
Example 4: Parametric Windows

Plot three parametric windows and their Fourier transform. Center the windows so that they have even symmetry and length $N = 25$ samples. Pick a wide, uniform range of 5 values for the parameter for each window.

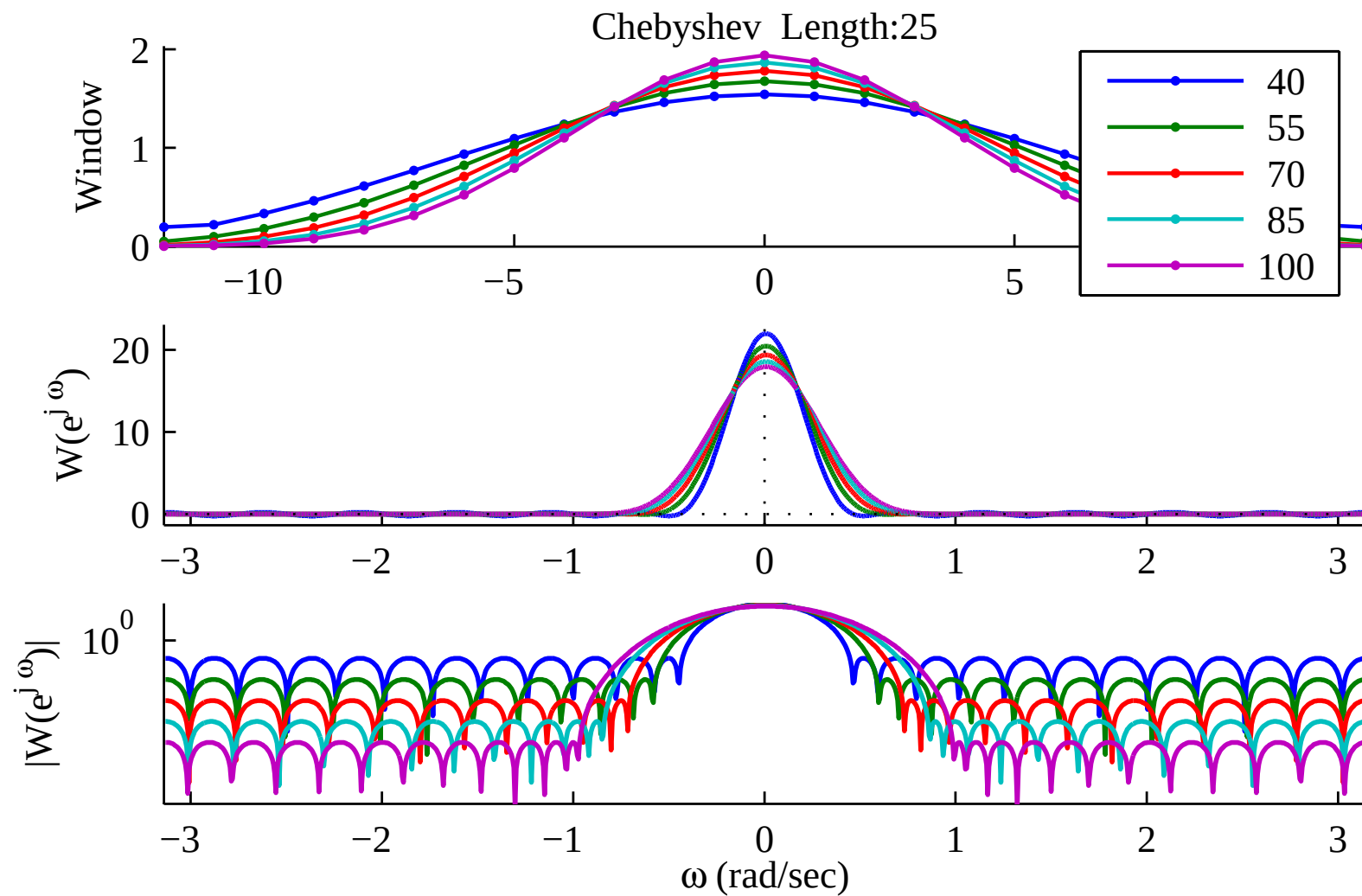
Example 4: Kaiser



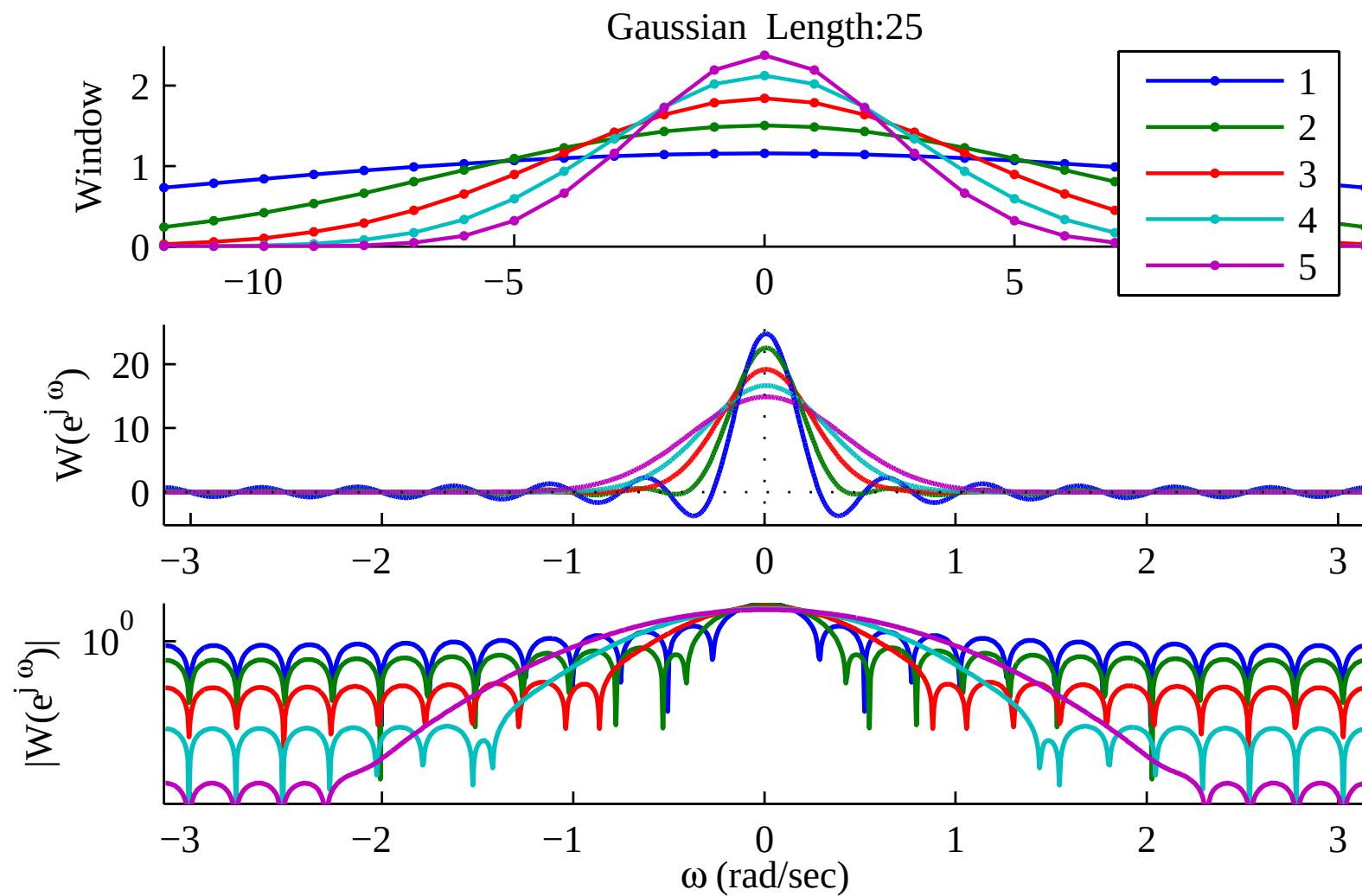
Example 4: Tukey



Example 4: Dolph-Chebyshev



Example 4: Gaussian



Example 4: MATLAB Code

```
%function [] = Windows;
close all;
clear;
N = 25;
M = 2^10;
np = 5;

%=====
% Main Loop
%=====

WA = zeros(M,np);
wA = zeros(N,np);

n = -(N-1)/2:(N-1)/2;
for c1 = 1:4,
    switch c1,
        case 1,
            pr = linspace(0,10,np);
            stW = 'Kaiser';
        case 2,
            pr = linspace(0,1,np);
            stW = 'Tukey';
        case 3,
            pr = linspace(40,100,np);
            stW = 'Chebyshev';
        case 4,
            pr = linspace(1,5,np);
```

```

        stW = 'Gaussian';
    end;
for c2=1:length(pr),
    switch c1,
        case 1, wn = kaiser(N,pr(c2));
        case 2, wn = tukeywin(N,pr(c2));
        case 3, wn = chebwin(N,pr(c2));
        case 4, wn = gausswin(N,pr(c2));
    end;
    wn = wn*sqrt(N/sum(wn.^2));
    w = (0:M-1)'*pi/(M/2);
    W = exp(j*w*(N-1)/2).*fft(wn,M);
    W = real(W);
    id = [M/2+1:M 1:M/2];
    W = W(id);
    w = -(M-3)/2:(M+1)/2;
    w = w*pi/(M/2);

    wA(:,c2) = wn;
    WA(:,c2) = W;
end;

figure;
FigureSet(1,'LTX');
subplot(3,1,1);
h = plot(n,wA);
set(h,'Marker','.');
set(h,'MarkerSize',6);
set(h,'LineWidth',0.8);
box off;
ylabel('Window');

```

```

        title(sprintf('%s Length:%d',stW,N));
        xlim((N-1)/2*[-1 1]);
        ylim([0 1.05*max(wn)]);
        legend(num2str(pr'))
subplot(3,1,2);
    h = plot(w,WA);
    set(h,'LineWidth',1.0);
    ylabel('W(e^{j \omega})');
    xlim([-pi pi]);
    ymn = min(min(WA));
    ymx = max(max(WA));
    yrg = ymx-ymn;
    ylim([ymn-0.05*yrg ymx+0.05*yrg]);
    AxisLines;
    box off;
subplot(3,1,3);
    h = semilogy(w,abs(WA));
    set(h,'LineWidth',1.0);
    ylabel('|W(e^{j \omega})|');
    xlabel('\omega (rad/sec)');
    xlim([-pi pi]);
    ymn = min(min(WA));
    ymx = max(max(WA));
    yrg = ymx-ymn;
    ylim([1e-6 1.05*ymx]);
    box off;
AxisSet(8);
    print(['Window' stW],'-depsc');
end;

return;

```

```

%=====
% All
%=====
[hrc,w] = freqz(ones(N,1)/N,1,2^10);
[htr,w] = freqz(triang(N)/sum(triang(N)),1,2^10);
[hhm,w] = freqz(hamming(N)/sum(hamming(N)),1,2^10);
[hhn,w] = freqz(hanning(N)/sum(hanning(N)),1,2^10);
[hbm,w] = freqz(blackman(N)/sum(blackman(N)),1,2^10);
[hbh,w] = freqz(blackmanharris(N)/sum(blackmanharris(N)),1,2^10);

figure;
FigureSet(1,'LTX');
f = w/(2*pi);
h = semilogy(f,abs(hrc),f,abs(htr),f,abs(hhm),f,abs(hhn),f,abs(hbm),f,abs(hbh));
set(h,'LineWidth',1.0);
ylim([1e-6 1]);
box off;
xlabel('Frequency (normalized)');
ylabel('H(e^{j\omega})');
title(sprintf('No. Points:%d',N));
AxisSet;
legend('Rectangular','Triangular','Hamming','Hanning','Blackman','Blackman Harris');
print -depsc WindowsLog;

figure;
FigureSet(1,'LTX');
f = w/(2*pi);
h = plot(f,abs(hrc),f,abs(htr),f,abs(hhm),f,abs(hhn),f,abs(hbm),f,abs(hbh));
set(h,'LineWidth',1.0);
ylim([0 1]);

```

```
box off;
xlabel('Frequency (normalized)');
ylabel('H(e^{j\omega})');
title(sprintf('No. Points:%d',N));
AxisSet;
legend('Rectangular','Triangular','Hamming','Hanning','Blackman','Blackman Harris');
print -depsc Windows;
```

Parametric Windows

- Kaiser
 - Tapered side lobes
 - No intuition for parameter
- Tukey
 - Tapered side lobes
 - Parameter controls rate of taper
- Dolph-Chebyshev
 - Flat side lobes (no taper)
 - Parameter controls degree of taper (dB)
- Gaussian
 - Tapered side lobes
 - Parameter controls width of main lobe

Summary

- Choice of the window controls the tradeoff between resolution and sidelobe leakage
- Rectangular window has maximum resolution and sidelobe leakage
- Some windows have parameters that permit you to control the tradeoff between resolution and leakage
- Both cause the spectral estimate to be biased
- Best window depends on actual spectrum, usually the user picks
- Ultimately, the window shape is much less important than the duration of the window
 - All windows converge to an impulse train in the limit as $N \rightarrow \infty$
 - There is a much larger gain in accuracy by increasing N than in picking the best window shape